# 5Klippa – Web App Runbook (Detailed)

Version: 1.0 • Audience: Engineers, QA, and Operators • Scope: Local dev, deployment, data model, rules, and end-to-end flows.

## Contents


1. Overview & Tech Stack
2. Repository Layout (expected)
3. Data Model (Collections & Key Fields)
4. Core Product Flows
5. Local Development (step-by-step)
6. Firebase Configuration (Project, CLI, Hosting)
7. Firestore Security Rules (full)
8. Build & Deploy (Hosting)
9. Test Plan / QA Checklist
10. Troubleshooting Guide
11. Operational Notes & Conventions

# 1) Overview & Tech Stack

5Klippa is a React (Vite) web application backed by Firebase Authentication, Firestore, and Firebase Hosting. It supports a single unified user profile for borrowing and lending, a multi-step onboarding flow, referral codes, a simple wallet, and a borrower loan workflow (apply & cancel).

## Key Technologies

```
- React + Vite (SPA, client-side routing via react-router-dom)
- Firebase Auth (Email/Password)
- Cloud Firestore (NoSQL database)
- Firebase Hosting (static hosting with SPA rewrites)
- Firebase CLI (deploy hosting & security rules)
```

# 2) Repository Layout (expected)

```
5klippa-web/
  ■■ src/
  ■   ■■ App.js
  ■   ■■ firebaseConfig.js
  ■   ■■ theme.js
  ■   ■■ components/
  ■   ■   ■■ AuthLayout.jsx
  ■   ■   ■■ DashboardLayout.jsx
  ■   ■■ utils/
  ■   ■   ■■ onboarding.js
  ■   ■■ pages/
  ■   ■   ■■ LoginPage.js
  ■   ■   ■■ RegisterPage.js
  ■   ■   ■■ DashboardPage.js
  ■   ■   ■■ LoansPage.jsx
  ■   ■   ■■ LoanDetailPage.jsx
  ■   ■   ■■ WalletPage.jsx
  ■   ■   ■■ ProfilePage.jsx
  ■   ■■ onboarding/
  ■       ■■ OnboardingLayout.jsx
  ■       ■■ steps/
  ■           ■■ AccountStep.jsx
  ■           ■■ IdentityStep.jsx
  ■           ■■ ContactStep.jsx
  ■           ■■ FinancialStep.jsx
  ■           ■■ RolesConsentStep.jsx
  ■           ■■ DoneStep.jsx
  ■■ index.html
  ■■ package.json
  ■■ vite.config.js
  ■■ firebase.json
  ■■ firestore.rules
```

# 3) Data Model (Collections & Key Fields)

## users/{uid}

```
- email: string
```

```
- fullName, idNumber, phone, addressLine1, addressLine2, city, postalCode
- bank: { accountNumber: string }
- creditConsent: boolean
- referralCodeUsed: string (doc ID of the referral used)
- referredBy: string | null (ownerUid who issued code)
- onboarding: {
    currentStep: 'identity' | 'contact' | 'financial' | 'roles' | 'done',
    completed: string[]
  }
- createdAt, updatedAt: timestamp
```

### applications/{applicationId}

```
- borrowerId: string (auth.uid)
- amount: number
- term: number (months)
- purpose: string
- status: 'pending' | 'approved' | 'rejected' | 'active' | 'cancelled'
- createdAt, cancelledAt: timestamp
```

### wallets/{uid}

```
- owner: string (uid)
- balance: number
- updatedAt: timestamp
```

### referrals/{CODE}

```
- ownerUid: string (who generated the code)
- maxUses: number (>=1, <=100)
- usedCount: number (starts at 0)
- active: boolean (owner can toggle)
- lastUsedAt: timestamp
(Note: The document ID IS THE CODE string itself, e.g. 'TEAM123'.)
```

### referralRedemptions/{refId}

```
- code: string (referrals doc id)
- userUid: string (who redeemed it)
- ownerUid: string | null (code owner)
- createdAt: timestamp
- (refId pattern often: `${code}_${userUid}` to enforce uniqueness)
```

# 4) Core Product Flows

## 4.1 Registration with Referral

```
- AccountStep.jsx (not logged in):
  1) User enters email, password, referral code.
  2) Client validates referral by reading `referrals/{CODE}`:
       - must exist
       - active == true
       - usedCount < maxUses
  3) Create auth user (createUserWithEmailAndPassword).
  4) In a Firestore transaction:
       - Re-read referral doc
```

```
      - Ensure still active and under maxUses
      - usedCount := usedCount + 1
      - write `referralRedemptions/{CODE_uid}` audit
  5) Create/merge user profile with `referralCodeUsed`, `referredBy`.
  6) Set onboarding.currentStep='identity', add 'account' to completed.
  7) Navigate to /onboarding/identity.
```

## 4.2 Generating Referral Codes (logged-in user)

```
- UI creates `referrals/{CODE}` where CODE is the exact code string.
- Fields: { ownerUid: uid, maxUses, usedCount: 0, active: true }
- Rules: only the owner can create their own code; public can read;
  any authed user can consume by incrementing usedCount by exactly 1 (transaction),
  but only the owner can toggle 'active'.
```

## 4.3 Onboarding Progress & Guard

```
- `nextOnboardingStep(userDoc)` infers the step if `onboarding.currentStep` is missing/stale.
- OnboardingLayout.jsx guards route access so a user only visits their current step (or 'accou
- Each step writes data and calls `markStepComplete()` to advance `onboarding.currentStep`.
```

## 4.4 Loans (Borrower)

```
- Create Application: adds to `applications` with borrowerId==auth.uid and status 'pending'.
- Cancel Application: only allowed by owner while status=='pending'; sets status 'cancelled' +
- Lists: Queries are always filtered by borrowerId==auth.uid.
```

## 4.5 Wallet

```
- `wallets/{uid}` balance is adjusted by the owner only.
- Top-up uses increment(100) or transaction; Withdraw uses a transaction to ensure non-negativ
```

# 5) Local Development (Step-by-Step)

```
Prerequisites
- Node.js 18+
- npm 9+
- Firebase CLI: npm i -g firebase-tools
- Access to the Firebase project (you must be added as a collaborator)

1. Clone and install
   git clone <your-repo-url>
   cd 5klippa-web
   npm install

2. Configure Firebase web app
   - Ensure src/firebaseConfig.js contains your project's config:
     export const firebaseConfig = { apiKey, authDomain, projectId, ... }
   - Ensure the app initializes:
     import { initializeApp } from 'firebase/app';
     const app = initializeApp(firebaseConfig);

3. Start Dev Server
   npm run dev
   # Typically runs on http://localhost:5173
```

```
4. Sign-in & Basic Checks
   - Register via onboarding with a valid referral code
   - Create a loan and cancel it while pending
   - Top-up/withdraw wallet (check Firestore writes)
```

# 6) Firebase Configuration (Project, CLI, Hosting)

```
1. Login & Select Project
   firebase login
   firebase use <your-project-id>

2. Ensure firebase.json (Vite SPA)
{
  "hosting": {
    "public": "dist",
    "ignore": ["firebase.json", "**/.*", "**/node_modules/**"],
    "rewrites": [{ "source": "**", "destination": "/index.html" }]
  }
}

3. Ensure firestore.rules exists at the repo root (see section 7).

4. (Optional) If you need to re-init
   firebase init hosting
   # choose: Use existing project, public=dist, SPA rewrite=Yes
```

# 7) Firestore Security Rules (Full)

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // ---- USERS ----
    match /users/{uid} {
      // Only the owner can read/write their profile/onboarding
      allow read: if request.auth != null && request.auth.uid == uid;
      allow create: if request.auth != null && request.auth.uid == uid;
      allow update: if request.auth != null && request.auth.uid == uid;
      allow delete: if false;
    }

    // ---- APPLICATIONS (Loans) ----
    match /applications/{id} {
      // Only the borrower can read their loan docs
      allow read: if request.auth != null && resource.data.borrowerId == request.auth.uid;

      // Create: borrowerId must match, minimal field validation
      allow create: if request.auth != null
        && request.resource.data.borrowerId == request.auth.uid
        && request.resource.data.amount is number
        && request.resource.data.term is int
        && request.resource.data.purpose is string;

      // Cancel by owner while 'pending' only; change only status + cancelledAt
      allow update: if request.auth != null
```

```
    && resource.data.borrowerId == request.auth.uid
    && resource.data.status == 'pending'
    && request.resource.data.diff(resource.data).changedKeys().hasOnly(['status','cancelle
    && request.resource.data.status == 'cancelled';

  allow delete: if false;
}

// ---- WALLETS ----
match /wallets/{uid} {
  allow read: if request.auth != null && request.auth.uid == uid;
  allow create: if request.auth != null && request.auth.uid == uid;
  // Owner can update their wallet doc (client uses transactions for safety)
  allow update: if request.auth != null && request.auth.uid == uid;
  allow delete: if false;
}

// ---- REFERRALS ----
match /referrals/{code} {
  // Public read so sign-up can validate a code
  allow read: if true;

  // Owner creates code doc; set initial usedCount=0, active=true, maxUses within range
  allow create: if request.auth != null
    && request.resource.data.ownerUid == request.auth.uid
    && request.resource.data.usedCount == 0
    && request.resource.data.maxUses is int
    && request.resource.data.maxUses >= 1
    && request.resource.data.maxUses <= 100
    && request.resource.data.active == true;

  // Two update paths:
  //  A) Any authed user can CONSUME exactly one usage (usedCount+1), nothing else changes
  //     and may not exceed maxUses.
  allow update: if request.auth != null
    && request.resource.data.ownerUid == resource.data.ownerUid
    && request.resource.data.maxUses == resource.data.maxUses
    && request.resource.data.active == resource.data.active
    && request.resource.data.usedCount == resource.data.usedCount + 1
    && request.resource.data.usedCount <= resource.data.maxUses

  //  B) Owner toggles ONLY 'active'
  || (request.auth != null
      && request.auth.uid == resource.data.ownerUid
      && request.resource.data.diff(resource.data).changedKeys().hasOnly(['active']));

  allow delete: if false;
}

// ---- REFERRAL REDEMPTIONS (Audit) ----
match /referralRedemptions/{rid} {
  // Owner or redeemer can read; write by redeemer only
  allow read: if request.auth != null
    && (request.auth.uid == resource.data.userUid
        || request.auth.uid == resource.data.ownerUid);
```

```
      allow create: if request.auth != null
        && request.resource.data.userUid == request.auth.uid;

      allow update, delete: if false;
    }

    // ---- Default deny ----
    match /{document=**} {
      allow read, write: if false;
    }
  }
}
```

# 8) Build & Deploy (Hosting)

```
1) Deploy Firestore Rules
   firebase deploy --only firestore:rules

2) Build the SPA
   npm run build

3) Deploy Hosting
   firebase deploy --only hosting

4) Verify
   - Open the hosting URL printed by the CLI
   - Register with a real referral code
   - Walk through onboarding and dashboard
```

# 9) Test Plan / QA Checklist

```
A. Registration & Onboarding
  [ ] New user can register with a valid referral {CODE}; invalid/disabled/exhausted codes are
  [ ] After registration, onboarding starts at Identity.
  [ ] Route guard prevents jumping ahead; can revisit already-completed steps.
  [ ] Profile completeness reflects saved fields; 'Continue onboarding' goes to the correct ne

B. Referrals
  [ ] Logged-in user can generate a code (referrals/{CODE}) with maxUses (1..100).
  [ ] Anyone can read code for validation; only owner can toggle 'active'.
  [ ] Consuming a code increments usedCount by exactly 1 via transaction; cannot exceed maxUse

C. Loans
  [ ] Create loan: shows under Loans list for that user.
  [ ] Cancel while pending: status becomes 'cancelled', cancel button disappears.
  [ ] Reading someone else’s loans is denied by rules.
  [ ] Dashboard snapshots reflect counts.

D. Wallet
  [ ] Top up +R100 updates balance; Withdraw -R100 never goes below zero.
  [ ] Only the owner can read/write their wallet.

E. Navigation & Layout
  [ ] Sidebar never scrolls; content area scrolls only.
  [ ] Profile initials show 1–2 letters (TE for “Test Example”).
```

```
F. Deployment
  [ ] Live site uses latest code (clear caches if needed).
  [ ] Rules deployed and effective (no “Missing or insufficient permissions” for legitimate fl
```

# 10) Troubleshooting Guide

## “Missing or insufficient permissions”

```
- Confirm you deployed rules recently: firebase deploy --only firestore:rules
- Use Firestore Rules Simulator with the same user auth context.
- Check collection/doc paths used in code match the rules (e.g., 'applications' vs 'applicatio
- For referrals: ensure docs are created with doc ID == code; consuming uses a transaction.
```

## Onboarding button loops back to Dashboard

```
- Check users/{uid}.onboarding.currentStep (must be one of: identity, contact, financial, role
- If inconsistent, OnboardingLayout reconciles with inferred step; reload the page after savin
```

## Referral generation works locally but not live

```
- Ensure rules are deployed to the live project you’re testing against (firebase use <projectI
- Verify the page writes referrals with the code as the document ID.
```

## Wallet not updating

```
- Confirm you’re authenticated (auth.currentUser) and writing to wallets/{uid}.
- For withdraw, ensure the transaction writes a non-negative balance.
```

# 11) Operational Notes & Conventions

```
- Referral codes are case-sensitive when used as doc IDs; standardize to UPPERCASE when genera
- Client code uses transactions for:
    * referral consumption (to avoid double-spend)
    * wallet withdrawal (prevent negative balances)
- Always query by borrowerId for applications; rules depend on it for read access.
- Keep SPA hosting rewrites enabled so deep links work (destination: /index.html).
- When changing rules, deploy them explicitly; rules are not bundled in app builds.
```

End of document • 5Klippa Runbook (Detailed)